

Apache Spark pour Big Data



Hmida HMIDA

24 février 2025



Introduction

Spark ?

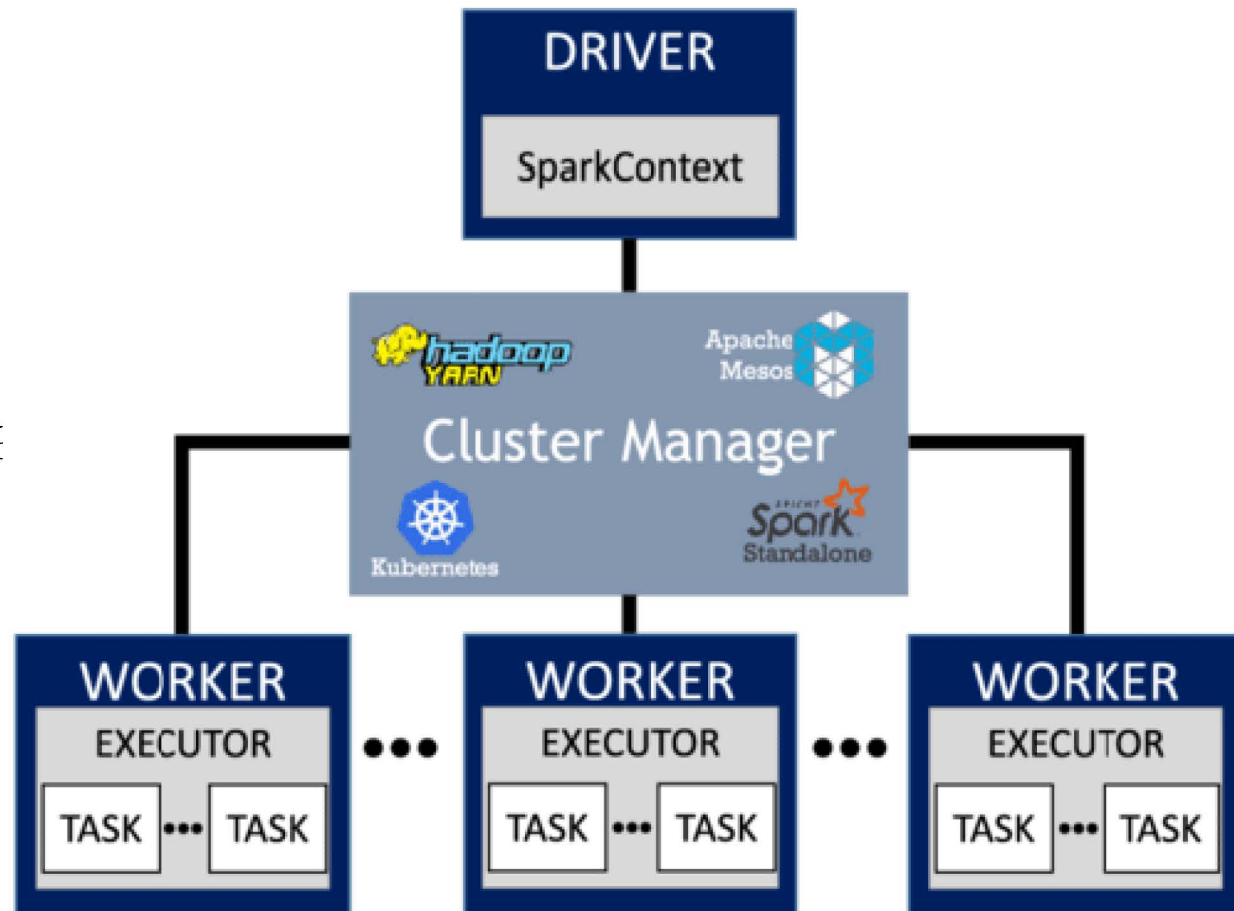


- Apache Spark™ is a multi-language engine for executing data engineering, data science, and machine learning on single-node machines or clusters.
 - ✓ Multi-langage : Scala, Java, Python, SQL, R
 - ✓ Moteur d'exécution : pas de stockage
 - ✓ Standalone ou Cluster : système distribué
 - ✓ Outils pour : ML, Streaming, Graph Analysis, ...
 - ✓ Implémente MapReduce et d'autres schémas
- Le moteur le plus utilisé pour l'analyse de données à grande échelle
- Inspiré de Hadoop mais :
 - ✓ 10x plus rapide – disque / 100x – mémoire
 - ✓ 2-5 x moins de code
- Créé en 2009
 - ✓ Matei Zaharia
 - ✓ UC Berkeley

Caractéristiques



- Cluster Manager
 - ✓ Local : single-node
 - ✓ Standalone : built-in
 - ✓ YARN : Hadoop
 - ✓ MESOS, Kubernetes
- In-memory
 - ✓ Mémoire distribuée
 - ✓ RDD : Resilient Distributed Dataset
- Master-Slave :
 - ✓ Cluster Spark
 - ▶ Spark master
 - ▶ Worker Nodes
 - ✓ Application Spark :
 - ▶ Driver Program : code Spark
 - ▶ Executor : JVM sur Worker

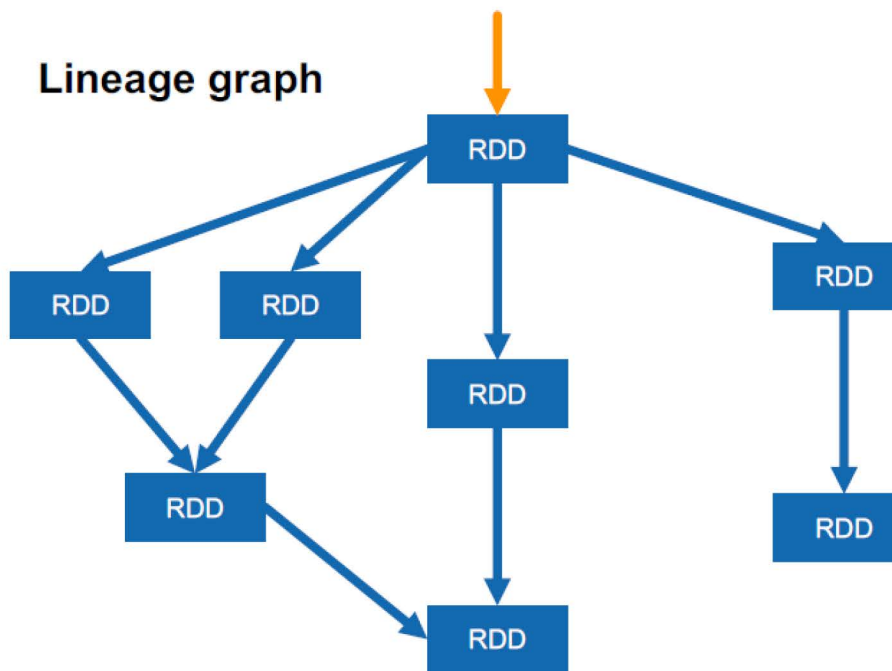


Caractéristiques (2)

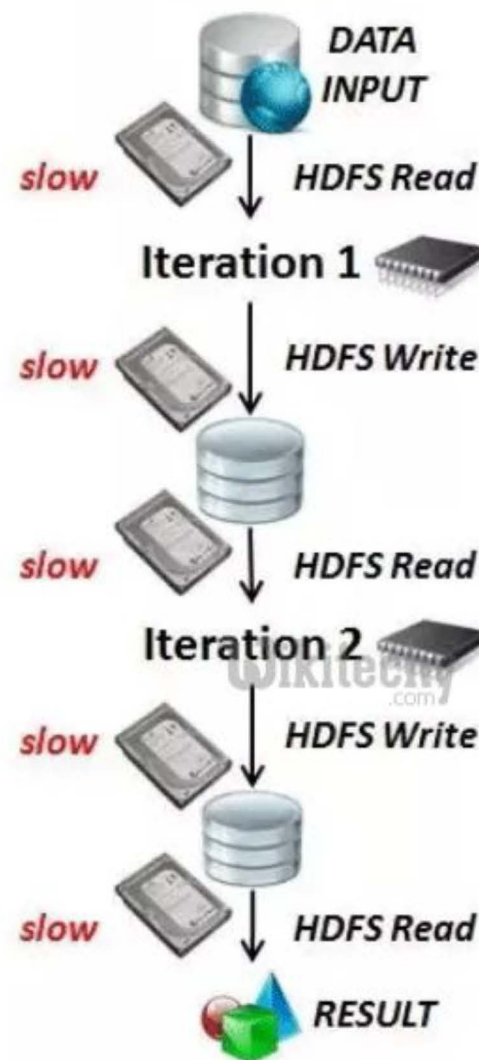
> Optimisations

- ✓ In-memory
- ✓ Lazy Evaluation
- ✓ DAG Scheduler

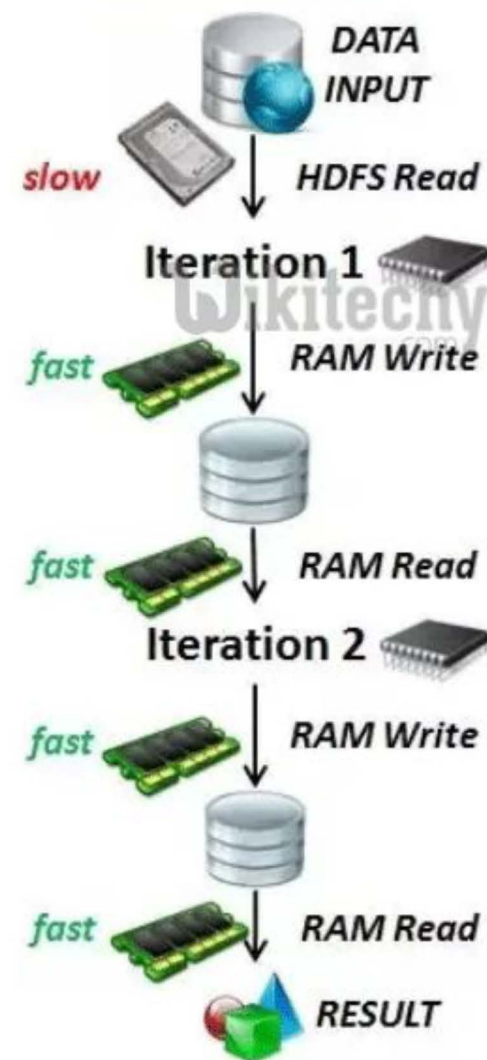
Lineage graph



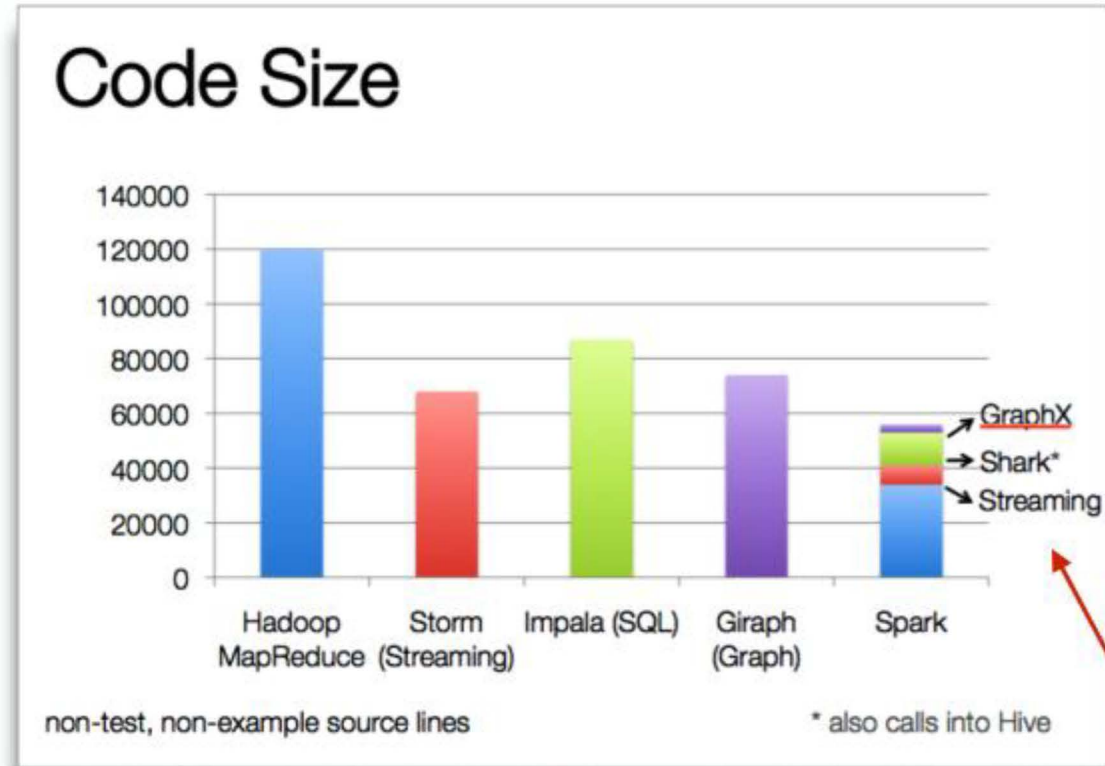
Apache Hadoop



Apache Spark



Caractéristiques (3)



The State of Spark, and Where We're Going Next
Matei Zaharia
Spark Summit (2013)
youtu.be/nU6vO2EJAb4

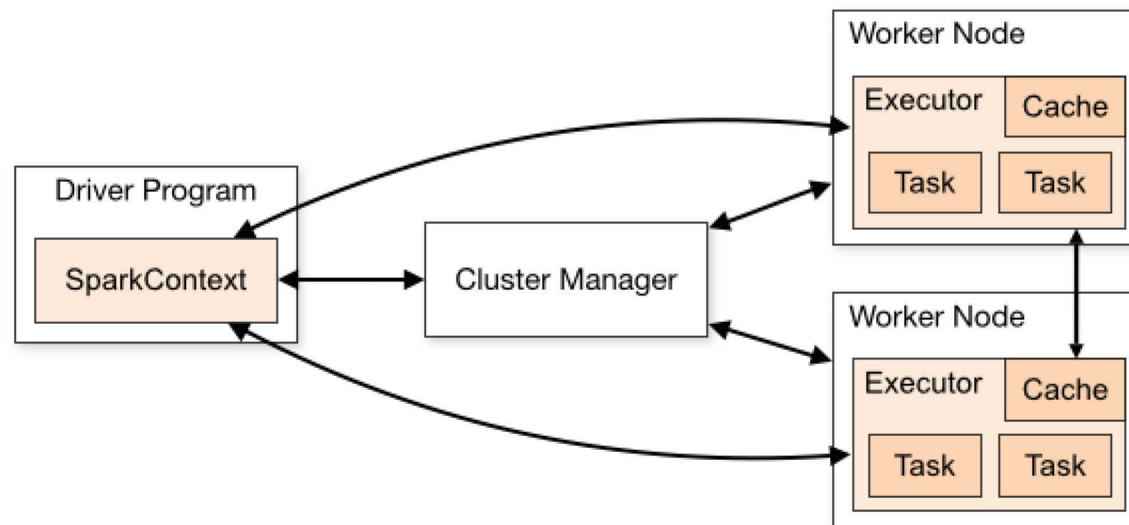
*used as libs, instead of
specialized systems*

Anatomie d'une application Spark

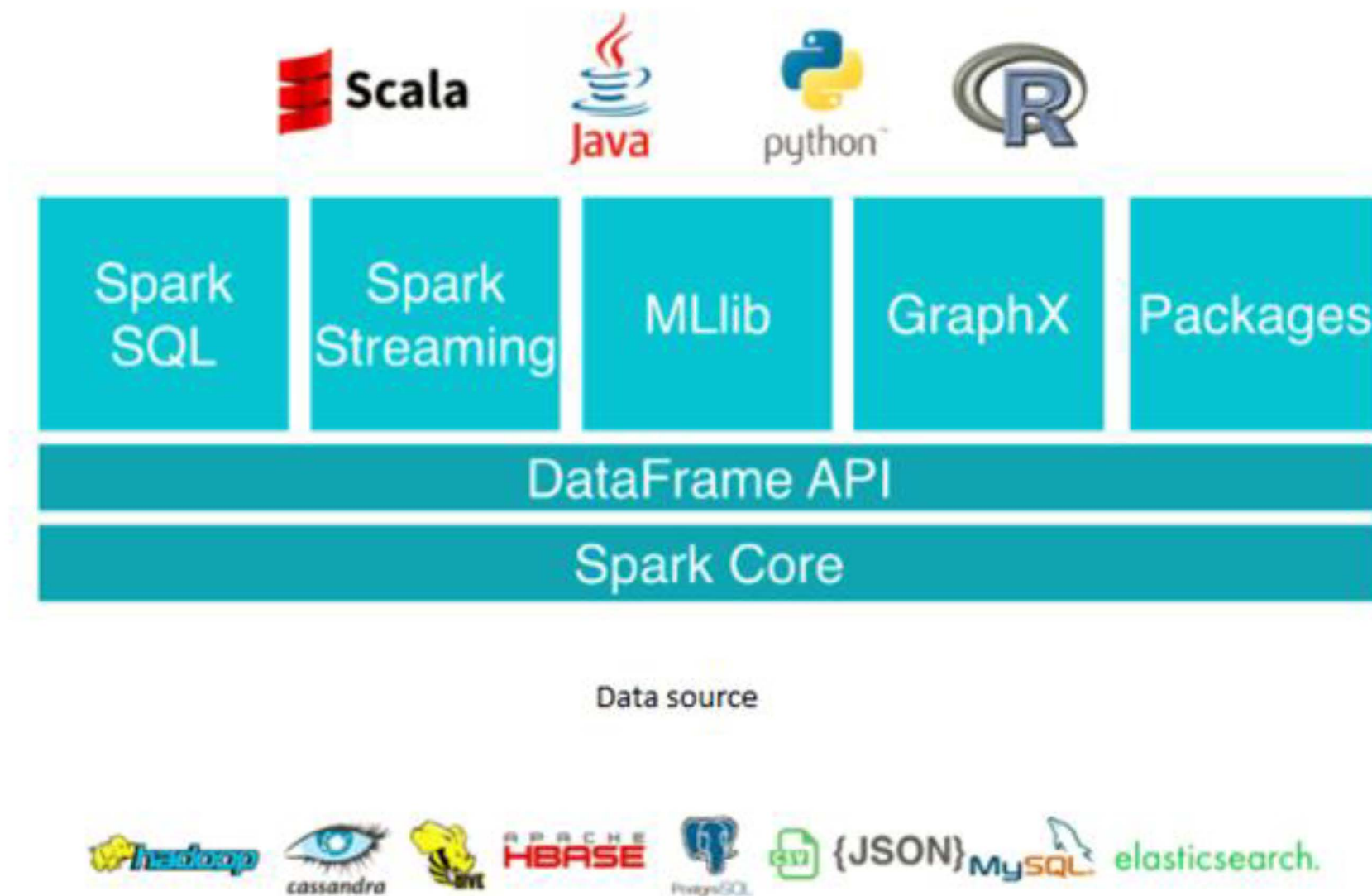


> Application Spark:

- ✓ Job : Un traitement parallèle en réponse à une action Spark (collect, save, ...)
- ✓ Stage : Chaque Job est découpé en étapes dépendantes similaires aux phases Map et Reduce.
- ✓ Task : Unité de travail envoyé à un Executor
- ✓ Driver Program : processus exécutant le programme principal
- ✓ Executor : Processus lancé sur un Worker Node, qui exécute des tâches pour le compte d'une application.



Framework Spark



Spark Core



- Spark Core est le composant principal de Spark.
- Spark Core définit les éléments suivants :
 - ✓ Les composants de base, tels que RDD et DataFrames
 - ✓ Les API disponibles pour effectuer des opérations sur ces abstractions de base
 - ✓ Variables partagées ou distribuées, telles que les variables de diffusion et les accumulateurs
- Spark Core définit également toutes les fonctionnalités de base, telles que :
 - ✓ La gestion des tâches,
 - ✓ La gestion de la mémoire,
 - ✓ Les fonctionnalités d'E / S de base, etc.

- Travailler avec des données structurées et semi-structurées
 - ✓ Les tables Hive, les tables MySQL, les fichiers Parquet, les fichiers AVRO, les fichiers JSON, les fichiers CSV, etc.
- Lorsqu'il existe des données structurées, SQL devient le premier choix de traitement de ces données.
- Spark SQL est le composant Spark qui permet aux développeurs de traiter les données avec le langage SQL.
- Spark fournit un shell SQL Spark, où vous pouvez exécuter les requêtes de type SQL et elles sont exécutées sur Spark.
- Spark convertit le code en une chaîne de calculs RDD.

Spark Streaming



- Spark Streaming est un package utilisé pour traiter un flux de données en temps réel.
- Spark Streaming utilise RDD et définit d'autres API pour traiter le flux de données en temps réel.
- Spark 2.x a introduit le streaming structuré, qui utilise des DataFrames plutôt que des RDD pour traiter le flux de données.



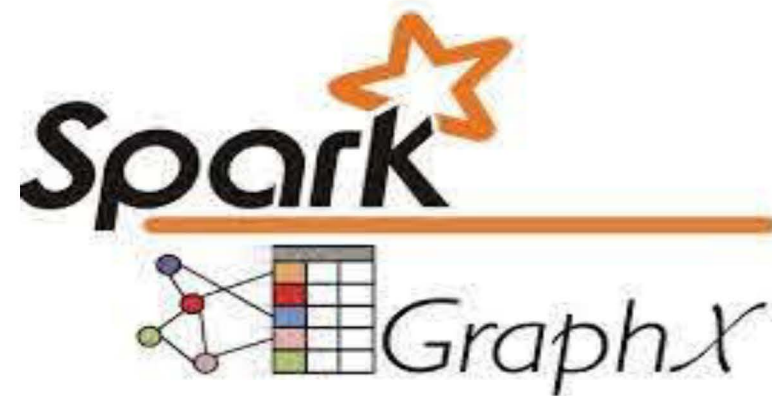
- La capacité de Spark à conserver une grande quantité de données en mémoire facilite l'écriture des algorithmes d'apprentissage automatique.
- Spark MLlib et ML sont les packages de Spark pour travailler avec des algorithmes d'apprentissage automatique.



Spark GraphX



- Un graphe se compose de sommets et d'arêtes
- Les arêtes définissent la relation entre les sommets
- Spark fournit GraphX pour traiter ces données
- GraphX utilise RDD pour son calcul



Écrire un programme Spark



> Interactif:

- ✓ Pyspark : shell pour Python, Spark-shell : shell pour Scala
 - ▶ Initialisation d'un SparkContext dans la variable **sc** et SparkSession dans **spark**
- ✓ Notebook Jupyter en mode local
 - ▶ Initialisation manuelle du SparkContext à partir de SparkSession
 - ▶ Initialisation avec findspark

> Soumission de jobs

- ✓ Spark-submit
- ✓ Source .py
- ✓ JAR (Maven, SBT) : Java et Scala

```
spark-submit \  
--master <master-url> \  
--deploy-mode <client|cluster> \  
--conf <key=value> \  
--driver-memory <value>g \  
--executor-memory <value>g \  
--executor-cores <number of cores> \  
--jars <comma separated dependencies> \  
--py-files <comma separated dependencies> \  
--class <main-class> \  
<application-jar-py> \  
[application-arguments]
```

SparkSession



- Point d'entrée unifié à Spark
 - ✓ Créer DataFrames, Datasets
 - ✓ Lecture/Écriture : CSV, JSON, Hive, Cassandra, ...
 - ✓ Métadonnées
 - ✓ Configuration Spark
- Accès à :
 - ✓ SQLContext,
 - ✓ SparkContext
 - ✓ StreamingContext
 - ✓ HiveContext
 - ✓ SparkConf

SparkSession –Example



```
from pyspark.sql import SparkSession
from pyspark.streaming import StreamingContext
from pyspark.sql import SQLContext
spark = SparkSession.builder \
    .master("local") \
    .appName("app name") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()
# Get SparkContext
sc = spark.sparkContext
# Get SQLContext
sqlc = SQLContext(sc)
# Get StreamingContext
streamc = StreamingContext(sc)
```

Applications



➤ Spark est un framework unifié pour

- ✓ Ingestion de données : ETL
- ✓ SQL Analytics
- ✓ Streaming
- ✓ Machine Learning
- ✓ Graph Analytics

➤ Cas d'utilisation

- ✓ Log Analysis : Ingestion + SQL
- ✓ Fraud Detection : Streaming + ML
- ✓ Community Detection : ML + Graph

